Assignment6 solution

1. ALSU UVM test with instantiated shift register

- **Top module**

```
import uvm_pkg::*;
`include "uvm_macros.svh"

import alsu_test_pkg::*;
import shared_pkg::*;
module top();
    //? clk generation
    bit clk;
    always #1 clk = ~clk;
    //? DUT & interface instantiation
    ALSU_if alsuif (clk);
    shift_reg_if shift_regif ();
    ALSU DUT (
        .clk(alsuif.clk),
        .cin(alsuif.cin),
        .rst(alsuif.rst),
        .red_op_A(alsuif.red_op_A),
        .red_op_B(alsuif.red_op_B),
        .bypass_A(alsuif.bypass_A),
        .bypass_B(alsuif.bypass_B),
        .direction(alsuif.direction),
        .serial_in(alsuif.serial_in),
        .opcode(alsuif.opcode),
        .A(alsuif.A),
        .B(alsuif.B),
        .leds(alsuif.leds),
        .out(alsuif.out)
    );
    //? internal shift reg interface assignment
    assign shift_regif.serial_in = DUT.serial_in_reg;
    assign shift_regif.direction = direction_e'(DUT.direction_reg);
    assign shift_regif.mode = mode_e'(DUT.opcode_reg[0]);
    assign shift_regif.datain = DUT.out;
    assign shift_regif.dataout = DUT.out_shift;
    //? UVM run_test task
    initial begin
        uvm_config_db #(virtual ALSU_if)::set(null, "uvm_test_top", "ALSU_IF", alsuif);
        uvm_config_db #(virtual shift_reg_if)::set(null, "uvm_test_top", "SHIFT_REG_IF", shift_regif);
        run_test("alsu_test");
    end

    bind ALSU alsu_sva alsu_sva_inst(alsuif.MONITOR);

endmodule
```

- **Shift register design**

```
module shift_reg (serial_in, direction, mode, datain, dataout);
input serial_in, direction, mode;
input [5:0] datain;
output reg [5:0] dataout;

always @(*) begin
    if (mode) // rotate
        if (direction) // left
            dataout <= {datain[4:0], datain[5]};
        else
            dataout <= {datain[0], datain[5:1]};
    else // shift
        if (direction) // left
            dataout <= {datain[4:0], serial_in};
        else
            dataout <= {serial_in, datain[5:1]};
end
endmodule
```

- **ALSU design**

```
module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER = "ON";
input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
input [2:0] opcode;
input signed [2:0] A, B;
output reg [15:0] leds;
output reg signed [5:0] out;

reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
reg signed [1:0] cin_reg;
reg [2:0] opcode_reg;
reg signed [2:0] A_reg, B_reg;

wire invalid_red_op, invalid_opcode, invalid;
wire [5:0] out_shift;

//instantiation
shift_reg SHFTREG (.serial_in(serial_in_reg),
                   .direction(direction_reg),
                   .mode(opcode_reg[0]),
                   .datain(out),
                   .dataout(out_shift));

//Invalid handling
assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
assign invalid = invalid_red_op | invalid_opcode;

//Registering input signals
always @(posedge clk or posedge rst) begin
  if(rst) begin
     cin_reg <= 0;
     red_op_B_reg <= 0;
     red_op_A_reg <= 0;
     bypass_B_reg <= 0;
     bypass_A_reg <= 0;
     direction_reg <= 0;
     serial_in_reg <= 0;
     opcode_reg <= 0;
     A_reg <= 0;
     B_reg <= 0;
  end else begin
     cin_reg <= cin;
     red_op_B_reg <= red_op_B;
     red_op_A_reg <= red_op_A;
     bypass_B_reg <= bypass_B;
     bypass_A_reg <= bypass_A;
     direction_reg <= direction;
     serial_in_reg <= serial_in;
     opcode_reg <= opcode;
     A_reg <= A;
     B_reg <= B;
  end
end

//leds output blinking
always @(posedge clk or posedge rst) begin
  if(rst) begin
     leds <= 0;
  end else begin
      if (invalid && !bypass_A_reg && !bypass_B_reg)
        leds <= ~leds;
      else
        leds <= 0;
  end
end

//ALSU output processing
always @(posedge clk or posedge rst) begin
  if(rst) begin
    out <= 0;
  end
  else begin
    if (bypass_A_reg && bypass_B_reg)
      out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
    else if (bypass_A_reg)
      out <= A_reg;
    else if (bypass_B_reg)
      out <= B_reg;
    else if (invalid)
      out <= 0;
    else begin
        case (opcode_reg)
          3'h0: begin
            if (red_op_A_reg && red_op_B_reg)
              out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
            else if (red_op_A_reg)
              out <= |A_reg;
            else if (red_op_B_reg)
              out <= |B_reg;
            else
              out <= A_reg | B_reg;
          end
          3'h1: begin
            if (red_op_A_reg && red_op_B_reg)
```

```
                out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
              else if (red_op_A_reg)
                out <= ^A_reg;
              else if (red_op_B_reg)
                out <= ^B_reg;
              else
                out <= A_reg ^ B_reg;
            end
            3'h2: out <= A_reg + B_reg + cin_reg;
            3'h3: out <= A_reg * B_reg;
            3'h4: out <= out_shift;
            3'h5: out <= out_shift;
          endcase
      end
    end
  end

endmodule
```

- **Shared package**

```
package shared_pkg;
  parameter INPUT_PRIORITY = "A";
  parameter FULL_ADDER = "ON";
  parameter MAXPOS = 3'b011;
  parameter MAXNEG = 3'b100;
  parameter ZERO = 3'b000;
  typedef enum logic [2:0] {
    OR,
    XOR,
    ADD,
    MULT,
    SHIFT,
    ROTATE,
    INVALID_6,
    INVALID_7
  } opcode_e;
  typedef enum logic {
    SHFT,
    ROT
  } mode_e;
  typedef enum logic {
    RIGHT,
    LEFT
  } direction_e;
endpackage
```

- **ALSU interface**

```
import shared_pkg::*;
interface ALSU_if (clk);
  input clk;
  logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
  opcode_e opcode;
  logic signed [2:0] A, B;
  logic [15:0] leds;
  logic signed [5:0] out;

  modport MONITOR(
      input clk,
            cin,
            rst,
            red_op_A,
            red_op_B,
            bypass_A,
            bypass_B,
            direction,
            serial_in,
            opcode,
            A,
            B,
            leds,
            out
  );
endinterface
```

- **Shift_reg interface**

```
import shared_pkg::*;
interface shift_reg_if ();
  logic serial_in;
  direction_e direction;
  mode_e mode;
  logic [5:0] datain, dataout;
endinterface : shift_reg_if
```

- **Alsu sequence item**

```
package alsu_seq_item_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    `include "uvm_macros.svh"

    class alsu_seq_item extends uvm_sequence_item;
        `uvm_object_utils(alsu_seq_item)

        rand bit cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
        rand opcode_e opcode;
        rand bit [2:0] A,B;
        logic [15:0] leds;
        logic [5:0] out;

        function new(string name = "alsu_seq_item");
            super.new(name);
        endfunction

        constraint alsu_constraints {
            rst dist {0:=99, 1:=1};
            bypass_A dist {0:=99, 1:=1};
            bypass_B dist {0:=99, 1:=1};
            opcode dist {INVALID_6:=10, INVALID_7:=10, [OR:ROTATE]:=90};
            if(opcode == OR || opcode == XOR){
                red_op_A dist {1:=30, 0:=70};
                red_op_B dist {1:=30, 0:=70};
                if(red_op_A) {
                    $onehot(A);
                    B == 0;
                } else if(red_op_B){
                    $onehot(B);
                    A == 0;
                }
            } else if(opcode == ADD || opcode == MULT) {
                A dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, {3'd1,3'd2,3'd5,3'd6,3'd7}:=20};
                B dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, {3'd1,3'd2,3'd5,3'd6,3'd7}:=20};
                red_op_A dist {1:=1, 0:=99};
                red_op_B dist {1:=1, 0:=99};
            } else {
                red_op_A dist {1:=1, 0:=99};
                red_op_B dist {1:=1, 0:=99};
            }
        }
        function string convert2string();
            return $sformatf("%s reset = 0b%0b, red_op_A = 0b%0b, red_op_B = %s, bypass_A = %s, bypass_B = 0b%0b, direction = 0b%0b, serial_in = 0b%0b, opcode = %s, A = 0b%0b, B = 0b%0b",
            super.convert2string(), rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in, opcode.name(), A, B);
        endfunction

        function string convert2string_stimulus();
            return $sformatf("reset = 0b%0b, red_op_A = 0b%0b, red_op_B = %s, bypass_A = %s, bypass_B = 0b%0b, direction = 0b%0b, serial_in = 0b%0b, opcode = %s, A = 0b%0b, B = 0b%0b",
            rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in, opcode.name(), A, B);
        endfunction
    endclass //alsu_seq_item extends superClass
endpackage
```

- **Shift_reg sequence item**

```
package shift_reg_seq_item_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    `include "uvm_macros.svh"
    class shift_reg_seq_item extends uvm_sequence_item;
        `uvm_object_utils(shift_reg_seq_item)

        rand bit serial_in;
        rand mode_e mode;
        rand direction_e direction;
        rand bit [5:0] datain;
        logic [5:0] dataout;

        function new(string name = "shift_reg_seq_item");
            super.new(name);
        endfunction

        function string convert2string();
            return $sformatf("%s, serial_in = 0b%0b, mode = %s, direction = %s, datain = 0b%0b, dataout = 0b%0b",
            super.convert2string(), serial_in, mode.name(), direction.name(), datain, dataout);
        endfunction

        function string convert2string_stimulus();
            return $sformatf("serial_in = 0b%0b, mode = %s, direction = %s, datain = 0b%0b, dataout = 0b%0b",
            serial_in, mode.name(), direction.name(), datain, dataout);
        endfunction

    endclass //shift_reg_seq_item extends superClass
endpackage
```

- **Alsu sequence**

```
package  alsu_seq_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    import alsu_seq_item_pkg::*;
    `include "uvm_macros.svh"

    class alsu_reset_seq extends uvm_sequence #(alsu_seq_item);
        `uvm_object_utils(alsu_reset_seq)
        alsu_seq_item seq_item;

        function new(string name = "alsu_seq");
            super.new(name);
        endfunction

        task body;
            seq_item = alsu_seq_item::type_id::create("seq_item");
            start_item(seq_item);
            seq_item.rst = 1;
            seq_item.cin = 0;
            seq_item.serial_in = 0;
            seq_item.direction = 0;
            seq_item.red_op_A = 0;
            seq_item.red_op_B = 0;
            seq_item.bypass_A = 0;
            seq_item.bypass_B = 0;
            seq_item.opcode = opcode_e'($random);
            seq_item.A = $random;
            seq_item.B = $random;
            finish_item(seq_item);
        endtask
    endclass //shift_reg_seq extends superClass

    class alsu_main_seq extends uvm_sequence #(alsu_seq_item);
        `uvm_object_utils(alsu_main_seq)
        alsu_seq_item seq_item;

        function new(string name = "alsu_seq_item");
            super.new(name);
        endfunction

        task body;
            seq_item = alsu_seq_item::type_id::create("seq_item");
            repeat(10000) begin
                start_item(seq_item);
                assert(seq_item.randomize());
                finish_item(seq_item);
            end
        endtask
    endclass //shift_reg_seq extends superClass
endpackage
```

- **Shift_reg sequence**

```
package shift_reg_seq_pkg;
    import uvm_pkg::*;
    import shared_pkg::*;
    import shift_reg_seq_item_pkg::*;
    `include "uvm_macros.svh"

    class shift_reg_main_seq extends uvm_sequence #(shift_reg_seq_item);
        `uvm_object_utils(shift_reg_main_seq)
        shift_reg_seq_item seq_item;

        function new(string name = "shift_reg_main_seq");
            super.new(name);
        endfunction

        task body;
            seq_item = shift_reg_seq_item::type_id::create("seq_item");
            repeat(10000) begin
                start_item(seq_item);
                assert(seq_item.randomize());
                finish_item(seq_item);
            end
        endtask
    endclass //shift_reg_seq extends superClass


endpackage
```

- **Alsu sequencer**

```
package alsu_sqr_pkg;
  import uvm_pkg::*;
  import alsu_seq_item_pkg::*;
  `include "uvm_macros.svh"

  class alsu_sqr extends uvm_sequencer #(alsu_seq_item);
    `uvm_component_utils(alsu_sqr)

    function new(string name = "alsu_sqr", uvm_component parent = null);
      super.new(name, parent);
    endfunction

  endclass

endpackage
```

- **Shift_reg sequencer**

```
package shift_reg_sqr_pkg;
  import uvm_pkg::*;
  import shift_reg_seq_item_pkg::*;
  `include "uvm_macros.svh"

  class shift_reg_sqr extends uvm_sequencer #(shift_reg_seq_item);
    `uvm_component_utils(shift_reg_sqr)

    function new(string name = "shift_reg_sqr", uvm_component parent = null);
      super.new(name, parent);
    endfunction

  endclass

endpackage
```

- **Alsu configuration object**

```
package alsu_config_obj_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

    class alsu_config_obj extends uvm_object;
        `uvm_object_utils(alsu_config_obj)

        virtual ALSU_if alsu_vif;
        uvm_active_passive_enum is_active;

        function new(string name = "alsu_config_obj");
            super.new(name);
        endfunction

    endclass
endpackage
```

- **Shift_reg configuration object**

```
package shift_reg_config_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"

  class shift_reg_config extends uvm_object;
    `uvm_object_utils(shift_reg_config)
    virtual shift_reg_if shift_reg_vif;
    uvm_active_passive_enum is_active;

    function new(string name = "shift_reg_config");
      super.new(name);
    endfunction
  endclass
endpackage
```

- **Alsu driver**

```
package shift_reg_driver_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import shift_reg_seq_item_pkg::*;

  class shift_reg_driver extends uvm_driver #(shift_reg_seq_item);
    `uvm_component_utils(shift_reg_driver)
    virtual shift_reg_if shift_reg_vif;
    shift_reg_seq_item stim_seq_item;

    function new(string name = "shift_reg_driver", uvm_component parent = null);
        super.new(name,parent);
    endfunction

    task run_phase(uvm_phase phase);
      super.run_phase(phase);
      stim_seq_item = shift_reg_seq_item::type_id::create("stim_seq_item");
      forever begin
        seq_item_port.get_next_item(stim_seq_item);
        shift_reg_vif.serial_in = stim_seq_item.serial_in;
        shift_reg_vif.direction = stim_seq_item.direction;
        shift_reg_vif.mode = stim_seq_item.mode;
        shift_reg_vif.datain = stim_seq_item.datain;
        #2;
        seq_item_port.item_done();
        `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
      end
    endtask
  endclass

endpackage
```

- **Alsu monitor**

```
package alsu_monitor_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import alsu_seq_item_pkg::*;

  class alsu_monitor extends uvm_monitor;
    `uvm_component_utils(alsu_monitor)
    virtual ALSU_if alsu_vif;
    alsu_seq_item rsp_seq_item;
    uvm_analysis_port #(alsu_seq_item) mon_ap;

    function new(string name = "alsu_monitor", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    // Build the driver in the build phase
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
  endfunction

    task run_phase(uvm_phase phase);
      super.run_phase(phase);
      forever begin
        rsp_seq_item = alsu_seq_item::type_id::create("rsp_seq_item");
        @(negedge alsu_vif.clk);
        @(negedge alsu_vif.clk);
        rsp_seq_item.rst = alsu_vif.rst;
        rsp_seq_item.cin = alsu_vif.cin;
        rsp_seq_item.serial_in = alsu_vif.serial_in;
        rsp_seq_item.direction = alsu_vif.direction;
        rsp_seq_item.red_op_A = alsu_vif.red_op_A;
        rsp_seq_item.red_op_B = alsu_vif.red_op_B;
        rsp_seq_item.bypass_A = alsu_vif.bypass_A;
        rsp_seq_item.bypass_B = alsu_vif.bypass_B;
        rsp_seq_item.opcode = alsu_vif.opcode;
        rsp_seq_item.A = alsu_vif.A;
        rsp_seq_item.B = alsu_vif.B;
        rsp_seq_item.out = alsu_vif.out;
        rsp_seq_item.leds = alsu_vif.leds;
        mon_ap.write(rsp_seq_item);
        `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
      end
    endtask
  endclass

endpackage
```

- **Shift_reg monitor**

```
package shift_reg_monitor_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import shift_reg_seq_item_pkg::*;

  class shift_reg_monitor extends uvm_monitor;
    `uvm_component_utils(shift_reg_monitor)
    virtual shift_reg_if shift_reg_vif;
    shift_reg_seq_item rsp_seq_item;
    uvm_analysis_port #(shift_reg_seq_item) mon_ap;

    function new(string name = "shift_reg_monitor", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    // Build the driver in the build phase
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
  endfunction

    task run_phase(uvm_phase phase);
      super.run_phase(phase);
      forever begin
        rsp_seq_item = shift_reg_seq_item::type_id::create("rsp_seq_item");
        #2;
        rsp_seq_item.serial_in = shift_reg_vif.serial_in;
        rsp_seq_item.direction = shift_reg_vif.direction;
        rsp_seq_item.mode = shift_reg_vif.mode;
        rsp_seq_item.datain = shift_reg_vif.datain;
        mon_ap.write(rsp_seq_item);
        `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
      end
    endtask
  endclass

endpackage
```

- **Alsu agent**

```
package alsu_agent_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import alsu_seq_item_pkg::*;
  import alsu_sqr_pkg::*;
  import alsu_driver_pkg::*;
  import alsu_monitor_pkg::*;
  import alsu_config_obj_pkg::*;

  class alsu_agent extends uvm_agent;
    `uvm_component_utils(alsu_agent)
    alsu_sqr sqr;
    alsu_driver drv;
    alsu_monitor mon;
    alsu_config_obj alsu_cfg;
    uvm_analysis_port #(alsu_seq_item) agt_ap;

    function new(string name = "alsu_agent", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon = alsu_monitor::type_id::create("mon", this);
        alsu_cfg = alsu_config_obj::type_id::create("alsu_cfg");
        agt_ap = new("agt_ap", this);
        if(!uvm_config_db #(alsu_config_obj)::get(this,"","ALSU_CFG",alsu_cfg)) begin
            `uvm_fatal("build_phase", "TEST - unable to get configuration object")
        end

        if(alsu_cfg.is_active == UVM_ACTIVE) begin
            sqr = alsu_sqr::type_id::create("sqr", this);
            drv = alsu_driver::type_id::create("drv", this);
        end

    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        if(alsu_cfg.is_active == UVM_ACTIVE) begin
            drv.alsu_vif = alsu_cfg.alsu_vif;
            drv.seq_item_port.connect(sqr.seq_item_export);
        end
        mon.alsu_vif = alsu_cfg.alsu_vif;
        mon.mon_ap.connect(agt_ap);
    endfunction
  endclass
endpackage
```

- **Shift_reg agent**

```
package shift_reg_agent_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import shift_reg_seq_item_pkg::*;
  import shift_reg_sqr_pkg::*;
  import shift_reg_driver_pkg::*;
  import shift_reg_monitor_pkg::*;
  import shift_reg_config_pkg::*;

  class shift_reg_agent extends uvm_agent;
    `uvm_component_utils(shift_reg_agent)
    shift_reg_sqr sqr;
    shift_reg_driver drv;
    shift_reg_monitor mon;
    shift_reg_config shift_cfg;
    uvm_analysis_port #(shift_reg_seq_item) agt_ap;

    function new(string name = "shift_reg_agent", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon = shift_reg_monitor::type_id::create("mon", this);
        shift_cfg = shift_reg_config::type_id::create("shift_cfg");
        agt_ap = new("agt_ap", this);

        if(!uvm_config_db #(shift_reg_config)::get(this,"","SHIFT_CFG",shift_cfg)) begin
            `uvm_fatal("build_phase", "TEST - unable to get configuration object")
        end

        if(shift_cfg.is_active == UVM_ACTIVE) begin
            sqr = shift_reg_sqr::type_id::create("sqr", this);
            drv = shift_reg_driver::type_id::create("drv", this);
        end
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        if(shift_cfg.is_active == UVM_ACTIVE) begin
            drv.shift_reg_vif = shift_cfg.shift_reg_vif;
            drv.seq_item_port.connect(sqr.seq_item_export);
        end
        mon.shift_reg_vif = shift_cfg.shift_reg_vif;
        mon.mon_ap.connect(agt_ap);
    endfunction
  endclass
endpackage
```

- **Shift_reg scoreboard**

```
package shift_reg_scoreboard_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import shift_reg_seq_item_pkg::*;

  class shift_reg_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(shift_reg_scoreboard)
    uvm_analysis_export #(shift_reg_seq_item) sb_export;
    uvm_tlm_analysis_fifo #(shift_reg_seq_item) sb_fifo;
    shift_reg_seq_item seq_item_sb;
    logic [5:0] dataout_ref;
    int error_count = 0;
    int correct_count = 0;
    function new(string name = "shift_reg_scoreboard", uvm_component parent = null);
        super.new(name, parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sb_export = new("sb_export", this);
        sb_fifo = new("sb_fifo", this);
    endfunction
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        sb_export.connect(sb_fifo.analysis_export);
    endfunction
    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            sb_fifo.get(seq_item_sb);
            ref_model(seq_item_sb);
            if(seq_item_sb.dataout != dataout_ref) begin
                `uvm_error("run_phase", "Comparison failed")
                error_count++;
            end
            else begin
                `uvm_info("run_phase", "Correct test result", UVM_HIGH)
                correct_count++;
            end
        end
    endtask
    task ref_model(shift_reg_seq_item seq_item_chk);
        if(seq_item_chk.mode)
            if(seq_item_chk.direction) dataout_ref = {seq_item_chk.datain[4:0], seq_item_chk.datain[5]};
            else dataout_ref = {seq_item_chk.datain[0], seq_item_chk.datain[5:1]};
        else if(seq_item_chk.direction) dataout_ref = {seq_item_chk.datain[4:0], seq_item_chk.serial_in};
        else dataout_ref = {seq_item_chk.serial_in, seq_item_chk.datain[5:1]};
    endtask
    function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        `uvm_info("report_phase", $sformatf("total successful transactions: %0d", correct_count), UVM_MEDIUM);
        `uvm_info("report_phase", $sformatf("total failed transactions: %0d", error_count), UVM_MEDIUM);
    endfunction
  endclass
endpackage
```

- **Alsu coverage collector**

```
package alsu_cover_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import alsu_seq_item_pkg::*;
  import shared_pkg::*;
  class alsu_cover extends uvm_component;
    `uvm_component_utils(alsu_cover)
    uvm_analysis_export #(alsu_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(alsu_seq_item) cov_fifo;
    alsu_seq_item seq_item_cov;
    covergroup cg;
        A_cp: coverpoint seq_item_cov.A {
                bins A_data_0 = {0};
                bins A_data_max = {MAXPOS};
                bins A_data_min = {MAXNEG};
                bins A_data_default = default;
            }
        A_data_cp: coverpoint seq_item_cov.A iff (seq_item_cov.red_op_A) {
            bins A_data_walkingones[] = {3'b001, 3'b010, 3'b100};
        }
        B_cp: coverpoint seq_item_cov.B {
            bins B_data_0 = {0};
            bins B_data_max = {MAXPOS};
            bins B_data_min = {MAXNEG};
            bins B_data_default = default;
        }
        B_data_cp: coverpoint seq_item_cov.B iff (seq_item_cov.red_op_B && !seq_item_cov.red_op_A){
            bins B_data_walkingones[] = {3'b001, 3'b010, 3'b100};
        }
        opcode_cp: coverpoint seq_item_cov.opcode {
            bins Bins_shift[] = {SHIFT,ROTATE};
            bins Bins_arith[] = {ADD,MULT};
            bins Bins_bitwise[] = {OR,XOR};
            illegal_bins Bins_invalid = {INVALID_6,INVALID_7};
        }
        cin_cp: coverpoint seq_item_cov.cin iff (seq_item_cov.opcode == ADD);
        direction_cp: coverpoint seq_item_cov.direction iff(seq_item_cov.opcode inside {SHIFT, ROTATE});
        serial_in_cp: coverpoint seq_item_cov.serial_in iff(seq_item_cov.opcode == SHIFT);
        red_opA_cp: coverpoint seq_item_cov.red_op_A {option.weight = 0;}
        red_opB_cp: coverpoint seq_item_cov.red_op_B {
            option.weight = 0;
            bins red_op_B0 = {0};
            bins red_op_B1 = {1};
        }
        ALU_cross_1_cp: cross A_cp, B_cp, opcode_cp {
            option.cross_auto_bin_max = 0;
            bins A_B_cross = binsof(A_cp) && binsof(B_cp) && binsof(opcode_cp.Bins_arith);
        }
        ALU_cross_2_cp: cross A_data_cp, B_cp iff(seq_item_cov.opcode inside {OR,XOR}){
            option.cross_auto_bin_max = 0;
            bins A_walking_cross_B = binsof(A_data_cp) && binsof(B_cp.B_data_0);
        }
```

```
        ALU_cross_3_cp: cross B_data_cp, A_cp, opcode_cp{
            option.cross_auto_bin_max = 0;
            bins B_walking_cross_A = binsof(B_data_cp) && binsof(A_cp.A_data_0) && binsof(opcode_cp) intersect {OR,XOR};
        }
        ALU_cross_4_cp: cross red_opA_cp, red_opB_cp iff (seq_item_cov.opcode != OR && seq_item_cov.opcode != XOR) {
            option.cross_auto_bin_max = 0;
            bins invalid_case = binsof(red_opA_cp) intersect {1} || binsof(red_opB_cp.red_op_B1);
        }
    endgroup
    function new(string name = "alsu_cover", uvm_component parent = null);
        super.new(name, parent);
        cg = new();
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        cov_export = new("cov_export", this);
        cov_fifo = new("cov_fifo", this);
    endfunction
    function void connect_phase(uvm_phase phase);
        super.build_phase(phase);
        cov_export.connect(cov_fifo.analysis_export);
    endfunction
    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            cov_fifo.get(seq_item_cov);
            cg.sample();
        end
    endtask
  endclass
endpackage
```

- **Shift_reg coverage collector**

```
package shift_reg_cover_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import shift_reg_seq_item_pkg::*;

  class shift_reg_cover extends uvm_component;
    `uvm_component_utils(shift_reg_cover)
    uvm_analysis_export #(shift_reg_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(shift_reg_seq_item) cov_fifo;
    shift_reg_seq_item seq_item_cov;

    covergroup cg;
        serial_in_cp: coverpoint seq_item_cov.serial_in;
        direction_cp: coverpoint seq_item_cov.direction;
        mode_cp: coverpoint seq_item_cov.mode;
        datain_cp: coverpoint seq_item_cov.datain;
    endgroup

    function new(string name = "shift_reg_cover", uvm_component parent = null);
        super.new(name, parent);
        cg = new();
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        cov_export = new("cov_export", this);
        cov_fifo = new("cov_fifo", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.build_phase(phase);
        cov_export.connect(cov_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            cov_fifo.get(seq_item_cov);
            cg.sample();
        end
    endtask
  endclass
endpackage
```

- **Alsu environment**

```
package alsu_env_pkg;
    import uvm_pkg::*;
    import alsu_cover_pkg::*;
    import alsu_agent_pkg::*;
    `include "uvm_macros.svh"

    class alsu_env extends uvm_env;
        `uvm_component_utils(alsu_env)

        alsu_agent agt;
        alsu_cover cov;

        function new(string name="alsu_env", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            agt = alsu_agent::type_id::create("agt",this);
            cov = alsu_cover::type_id::create("cov",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            agt.agt_ap.connect(cov.cov_export);
        endfunction
    endclass
endpackage
```

- **Shift_reg environment**

```
package shift_reg_env_pkg;
import shift_reg_scoreboard_pkg::*;
import shift_reg_cover_pkg::*;
import shift_reg_agent_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class shift_reg_env extends uvm_env;
  // Example 1 (Done for you)
  // Do the essentials (factory register & Constructor)
  `uvm_component_utils(shift_reg_env)
  shift_reg_agent agt;
  shift_reg_scoreboard sb;
  shift_reg_cover cov;

  function new(string name = "shift_reg_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  // Example 2
  // Build the driver in the build phase
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agt = shift_reg_agent::type_id::create("agt",this);
    sb = shift_reg_scoreboard::type_id::create("sb",this);
    cov = shift_reg_cover::type_id::create("cov",this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    agt.agt_ap.connect(sb.sb_export);
    agt.agt_ap.connect(cov.cov_export);
  endfunction
endclass
endpackage
```

- **Alsu test**

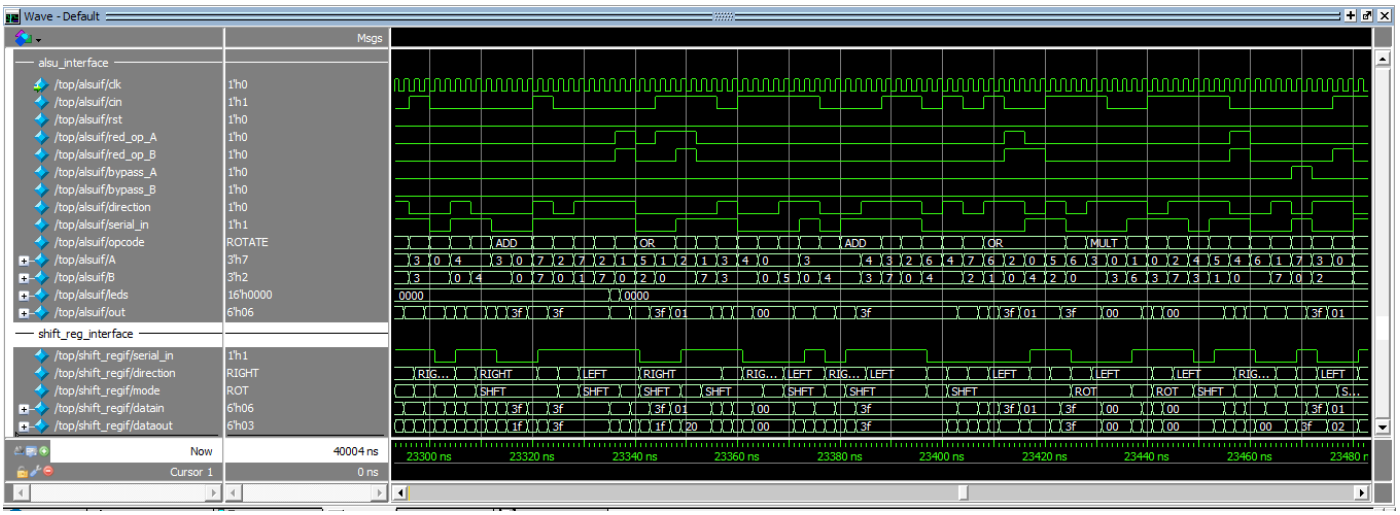
```
package alsu_test_pkg;
    import uvm_pkg::*;
    import alsu_env_pkg::*;
    import alsu_config_obj_pkg::*;
    import alsu_seq_pkg::*;
    import shift_reg_env_pkg::*;
    import shift_reg_config_pkg::*;
    `include "uvm_macros.svh"

    class alsu_test extends uvm_test;
        `uvm_component_utils(alsu_test)
        alsu_env alsu_env_obj;
        shift_reg_env shift_env;
        alsu_config_obj alsu_cfg;
        shift_reg_config shift_cfg;
        alsu_main_seq main_seq;
        alsu_reset_seq reset_seq;
        //? constructor
        function new(string name="alsu_test", uvm_component parent = null);
            super.new(name, parent);
        endfunction
        //? phases
        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            alsu_env_obj = alsu_env::type_id::create("alsu_env_obj",this);
            shift_env = shift_reg_env::type_id::create("shift_env", this);
            alsu_cfg = alsu_config_obj::type_id::create("alsu_cfg");
            shift_cfg = shift_reg_config::type_id::create("shift_cfg");
            main_seq = alsu_main_seq::type_id::create("main_seq");
            reset_seq = alsu_reset_seq::type_id::create("reset_seq");

            if(!uvm_config_db #(virtual ALSU_if)::get(this,"","ALSU_IF", alsu_cfg.alsu_vif))
                `uvm_fatal("build_phase", "TEST - unable to get virtual interface")
            if(!uvm_config_db #(virtual shift_reg_if)::get(this,"","SHIFT_REG_IF",shift_cfg.shift_reg_vif))
                `uvm_fatal("build_phase", "TEST - unable to get virtual interface")

            alsu_cfg.is_active = UVM_ACTIVE;
            shift_cfg.is_active = UVM_PASSIVE;

            uvm_config_db #(alsu_config_obj)::set(this,"*","ALSU_CFG",alsu_cfg);
            uvm_config_db #(shift_reg_config)::set(this,"*","SHIFT_CFG",shift_cfg);
        endfunction
        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            phase.raise_objection(this);
            `uvm_info("run_phase", "Reset asserted", UVM_LOW)
            reset_seq.start(alsu_env_obj.agt.sqr);
            `uvm_info("run_phase", "Reset deasserted", UVM_LOW)

            `uvm_info("run_phase", "main stimulus generation started", UVM_LOW)
            main_seq.start(alsu_env_obj.agt.sqr);
            `uvm_info("run_phase", "main stimulus generation ended", UVM_LOW)
            phase.drop_objection(this);
        endtask

    endclass
endpackage
```

- **Alsu assertions**

```
module alsu_sva(ALSU_if.MONITOR alsuif);
    import shared_pkg::*;
    bit invalid, normal_op;
    int correct_count, error_count;
    //? asynchronous reset assertion
    always_comb begin
        if(alsuif.rst)
            arst_sva: assert final(alsuif.leds == 0 && alsuif.out == 0);
    end
    assign invalid = ( (alsuif.red_op_A | alsuif.red_op_B) & (alsuif.opcode[1] | alsuif.opcode[2]) ) | (alsuif.opcode[1] & alsuif.opcode[2]);
    assign normal_op = ~alsuif.bypass_A & ~alsuif.bypass_B & ~invalid;
    //? PROPERTIES
    property invalid_leds_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) invalid && !alsuif.bypass_A && !alsuif.bypass_B |=>
                                                ##1 alsuif.leds == ~$past(alsuif.leds);
    endproperty

    property bypassA_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) alsuif.bypass_A |=>
                                                ##1 alsuif.out == $past(alsuif.A,2);
    endproperty

    property bypassB_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) alsuif.bypass_B && !alsuif.bypass_A |=>
                                                ##1 alsuif.out == $past(alsuif.B,2);
    endproperty

    property invalid_out_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) !alsuif.bypass_A && !alsuif.bypass_B && invalid |=> ##1 alsuif.out == 0;
    endproperty

    property red_ora_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == OR && alsuif.red_op_A |=>
                                                ##1 alsuif.out == |$past(alsuif.A,2);
    endproperty

    property red_orb_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == OR && alsuif.red_op_B && !alsuif.red_op_A |=>
                                                ##1 alsuif.out == |$past(alsuif.B,2);
    endproperty

    property btw_or_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == OR && !alsuif.red_op_A && !alsuif.red_op_B |=>
                                                ##1 alsuif.out == $past(alsuif.A,2) | $past(alsuif.B,2);
    endproperty

    property red_xora_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == XOR && alsuif.red_op_A |=>
                                                ##1 alsuif.out == ^$past(alsuif.A,2);
    endproperty

    property red_xorb_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == XOR && alsuif.red_op_B && !alsuif.red_op_A |=>
                                                ##1 alsuif.out == ^$past(alsuif.B,2);
    endproperty

    property btw_xor_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == XOR && !alsuif.red_op_A && !alsuif.red_op_B |=>
                                                ##1 alsuif.out == $past(alsuif.A,2) ^ $past(alsuif.B,2);
    endproperty

    property add_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == ADD |=>
                                                ##1 alsuif.out == $signed(6'($past(alsuif.A, 2))) + $signed(6'($past(alsuif.B, 2))) + $past(alsuif.cin, 2);
    endproperty

    property mult_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == MULT |=>
                                                ##1 alsuif.out == $past(alsuif.A,2) * $past(alsuif.B,2);
    endproperty

    property shift_left_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == SHIFT && alsuif.direction |=>
                                                ##1 alsuif.out == {$past(alsuif.out[4:0]), $past(alsuif.serial_in,2)};
    endproperty

    property shift_right_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == SHIFT && !alsuif.direction|=>
                                                ##1 alsuif.out == {$past(alsuif.serial_in,2), $past(alsuif.out[5:1])};
    endproperty

    property rotate_left_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == ROTATE && alsuif.direction|=>
                                                ##1 alsuif.out == {$past(alsuif.out[4:0]), $past(alsuif.out[5])};
    endproperty
```

```
    property rotate_right_sva_check;
        @(posedge alsuif.clk) disable iff(alsuif.rst) normal_op && alsuif.opcode == ROTATE && !alsuif.direction|=>
                                                    ##1 alsuif.out == {$past(alsuif.out[0]), $past(alsuif.out[5:1])};
    endproperty

    //? ASSERTIONS
    invalid_leds_sva:       assert property (invalid_leds_sva_check);
    bypassA_sva:            assert property (bypassA_sva_check);
    bypassB_sva:            assert property (bypassB_sva_check);
    invalid_out_sva:        assert property (invalid_out_sva_check);
    red_ora_sva:            assert property (red_ora_sva_check);
    red_orb_sva:            assert property (red_orb_sva_check);
    btw_or_sva:             assert property (btw_or_sva_check);
    red_xora_sva:           assert property (red_xora_sva_check);
    red_xorb_sva:           assert property (red_xorb_sva_check);
    add_sva:                assert property (add_sva_check);
    mult_sva:               assert property (mult_sva_check);
    shift_left_sva:         assert property (shift_left_sva_check);
    shift_right_sva:        assert property (shift_right_sva_check);
    rotate_left_sva:        assert property (rotate_left_sva_check);
    rotate_right_sva:       assert property (rotate_right_sva_check);
    //? COVER PROPERTIES
    invalid_leds_sva_cp:        cover property (invalid_leds_sva_check);
    bypassA_sva_cp:             cover property (bypassA_sva_check);
    bypassB_sva_cp:             cover property (bypassB_sva_check) ;
    invalid_out_sva_cp:         cover property (invalid_out_sva_check);
    red_ora_sva_cp:             cover property (red_ora_sva_check);
    red_orb_sva_cp:             cover property (red_orb_sva_check);
    btw_or_sva_cp:              cover property (btw_or_sva_check);
    red_xora_sva_cp:            cover property (red_xora_sva_check);
    red_xorb_sva_cp:            cover property (red_xorb_sva_check);
    add_sva_cp:                 cover property (add_sva_check);
    mult_sva_cp:                cover property (mult_sva_check);
    shift_left_sva_cp:          cover property (shift_left_sva_check);
    shift_right_sva_cp:         cover property (shift_right_sva_check);
    rotate_left_sva_cp:         cover property (rotate_left_sva_check);
    rotate_right_sva_cp:        cover property (rotate_right_sva_check);

endmodule
```

- **Simulation results**



- **Code coverage summary**

```
File  Edit  Format  View  Help
Coverage Report Summary Data by DU

================================================================================
=== Design Unit: work.ALSU
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        20        20         0   100.00%
    Statements                      46        46         0   100.00%
    Toggles                        130       130         0   100.00%


Total Coverage By Design Unit (filtered view): 100.00%
```

- **Functional coverage**

Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_ins
/alsu_cover_pkg/alsu_cover		100.00%						
TYPE cg		100.00%	100	100.00...		✓	auto(1)	
CVP cg::A_cp		100.00%	100	100.00...		✓		
CVP cg::A_data_cp		100.00%	100	100.00...		✓		
CVP cg::B_cp		100.00%	100	100.00...		✓		
CVP cg::B_data_cp		100.00%	100	100.00...		✓		
CVP cg::opcode_cp		100.00%	100	100.00...		✓		
CVP cg::cin_cp		100.00%	100	100.00...		✓		
CVP cg::direction_cp		100.00%	100	100.00...		✓		
CVP cg::serial_in_cp		100.00%	100	100.00...		✓		
CVP cg::red_opA_cp		100.00%	100	100.00...		✓		
CVP cg::red_opB_cp		100.00%	100	100.00...		✓		
CROSS cg::ALU_cross_1_cp		100.00%	100	100.00...		✓		
CROSS cg::ALU_cross_2_cp		100.00%	100	100.00...		✓		
CROSS cg::ALU_cross_3_cp		100.00%	100	100.00...		✓		
CROSS cg::ALU_cross_4_cp		100.00%	100	100.00...		✓		
/shift_reg_cover_pkg/shift_reg_cover		95.70%						
TYPE cg		95.70%	100	95.70%		✓	auto(1)	
CVP cg::serial_in_cp		100.00%	100	100.00...		✓		
CVP cg::direction_cp		100.00%	100	100.00...		✓		
CVP cg::mode_cp		100.00%	100	100.00...		✓		
CVP cg::datain_cp		82.81%	100	82.81%		✓		

- **Assertions**

Name	Assertion Type	Language	Enable	Failure Count	Pass Count
/uvm_pkg::uvm_reg_map::do_write/#ublk#215181159#1731/immed__1735	Immediate	SVA	on	0	0
/uvm_pkg::uvm_reg_map::do_read/#ublk#215181159#1771/immed__1775	Immediate	SVA	on	0	0
/alsu_seq_pkg::alsu_main_seq::body/#ublk#256298615#43/immed__45	Immediate	SVA	on	0	1
/top/DUT/alsu_sva_inst/arst_sva	Immediate	SVA	on	0	1
/top/DUT/alsu_sva_inst/invalid_leds_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/bypassA_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/bypassB_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/invalid_out_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/red_ora_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/red_orb_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/btw_or_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/red_xora_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/red_xorb_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/add_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/mult_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/shift_left_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/shift_right_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/rotate_left_sva	Concurrent	SVA	on	0	1
/top/DUT/alsu_sva_inst/rotate_right_sva	Concurrent	SVA	on	0	1

- **Assertion coverage**

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included
/top/DUT/alsu_sva_inst/invalid_leds_sva_cp	SVA	✓	Off	952	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/bypassA_sva_cp	SVA	✓	Off	192	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/bypassB_sva_cp	SVA	✓	Off	176	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/invalid_out_sva_cp	SVA	✓	Off	952	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/red_ora_sva_cp	SVA	✓	Off	968	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/red_orb_sva_cp	SVA	✓	Off	704	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/btw_or_sva_cp	SVA	✓	Off	1508	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/red_xora_sva_cp	SVA	✓	Off	898	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/red_xorb_sva_cp	SVA	✓	Off	596	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/add_sva_cp	SVA	✓	Off	2870	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/mult_sva_cp	SVA	✓	Off	3132	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/shift_left_sva_cp	SVA	✓	Off	1458	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/shift_right_sva_cp	SVA	✓	Off	1520	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/rotate_left_sva_cp	SVA	✓	Off	1544	1	Unli...	1	100%		✓
/top/DUT/alsu_sva_inst/rotate_right_sva_cp	SVA	✓	Off	1508	1	Unli...	1	100%		✓